

Práctica 1: gestión de clientes y repostajes de una gasolinera

Resultado de aprendizaje evaluado

RA1. Desarrolla aplicaciones que gestionan información almacenada en ficheros identificando el campo de aplicación de los mismos y utilizando clases específicas.

Contexto y objetivo

Una gasolinera necesita una aplicación de consola para registrar a sus clientes y los pagos correspondientes a sus repostajes. Los datos deben conservarse entre ejecuciones mediante ficheros de texto en formato **CSV**.

Desarrolla la aplicación en **Java**, utilizando la clase `java.nio.file.Files` para acceder a los ficheros. Además de funcionar correctamente, el programa debe resultar comprensible y permitir modificar una parte sin provocar cambios innecesarios en otras.

Este documento establece el comportamiento que debe observar el usuario. Las decisiones de organización del código se trabajarán en clase: deberás aplicar las indicaciones dadas durante esas sesiones y justificar tu solución. El enunciado no fija una estructura de clases.

Datos que se deben gestionar

Clientes

Cada cliente tendrá:

Campo	Requisitos
Identificador	Entero positivo, asignado automáticamente e invariable.
Nombre	Texto obligatorio.
Teléfono	Texto obligatorio; puede contener prefijos, espacios o ceros iniciales.
Matrícula	Texto obligatorio, almacenado en mayúsculas y único entre los clientes.

Se eliminarán los espacios al principio y al final de los textos introducidos. No es necesario comprobar que el teléfono o la matrícula se ajusten a un formato nacional concreto. Dos matrículas que solo difieran en mayúsculas o minúsculas se considerarán iguales.

Pagos de repostajes

Cada pago tendrá:

Campo	Requisitos
Identificador	Entero positivo asignado automáticamente.
Identificador del cliente	Debe corresponder a un cliente registrado.
Fecha	Fecha real del calendario, introducida como <code>dd/MM/aaaa</code> .
Importe	Cantidad en euros, finita y mayor que cero, con un máximo de dos decimales.
Litros	Cantidad finita y mayor que cero, con un máximo de dos decimales.
Combustible	Texto obligatorio, por ejemplo, <code>Gasolina 95</code> o <code>Diésel</code> .

El programa aceptará tanto coma como punto como separador decimal, sin separadores de miles. Al mostrar cantidades utilizará dos decimales. Si se deja la fecha vacía, utilizará la fecha actual.

En esta práctica, **procesar un pago significa registrar un repostaje ya abonado**. No se conecta con bancos, no se cobra mediante tarjeta y no se calculan deudas. El importe y los litros se introducen de forma independiente; no se calcula un precio por litro.

Funcionalidades obligatorias

1. Mostrar el menú

Al arrancar, la aplicación preparará el almacenamiento necesario y mostrará estas opciones:

```
=== GESTIÓN DE GASOLINERA ===
1. Dar de alta un cliente
2. Listar clientes
3. Buscar clientes
4. Procesar un pago de repostaje
```

5. Consultar pagos
0. Salir
Opción:

Después de completar o rechazar una operación, volverá al menú. Ante una opción desconocida, mostrará un mensaje y permitirá elegir de nuevo. La opción 0 finalizará el programa de forma ordenada.

2. Dar de alta un cliente

1. Solicitar nombre, teléfono y matrícula.
2. Repetir la petición de cualquier campo obligatorio que quede vacío.
3. Comprobar que la matrícula no esté registrada. Si está duplicada, informar y volver al menú sin guardar otro cliente.
4. Asignar un identificador, guardar el cliente y confirmar el resultado mostrando su identificador.

La numeración comenzará en 1 cuando no haya clientes. Los nuevos identificadores serán el máximo existente más uno y no se repetirán al reiniciar el programa.

3. Listar clientes

Mostrar identificador, nombre, teléfono y matrícula de todos los clientes, ordenados por nombre sin distinguir mayúsculas de minúsculas. Si dos nombres coinciden, ordenar por identificador ascendente. Si no hay clientes, indicarlo expresamente.

4. Buscar clientes

Solicitar un texto no vacío y buscar coincidencias parciales en el nombre, el teléfono o la matrícula, sin distinguir mayúsculas de minúsculas. Mostrar todos los resultados con los mismos campos y orden que en el listado general. Si no hay coincidencias, indicarlo.

No se exige que la búsqueda ignore las tildes.

5. Procesar un pago de repostaje

1. Si no hay clientes, informar de que primero debe darse de alta uno y volver al menú.
2. Mostrar los clientes disponibles y solicitar el identificador del cliente.

3. Si la entrada no es un entero positivo, solicitarla de nuevo. Si el identificador no corresponde a ningún cliente, informar y volver al menú sin registrar el pago.
4. Solicitar fecha, importe, litros y combustible. Repetir la petición del campo que no sea válido, explicando el motivo.
5. Asignar un identificador de pago, guardar la operación y mostrar una confirmación con el identificador, el nombre del cliente y el importe.

La numeración de pagos será independiente de la de clientes: comenzará en **1** y continuará a partir del máximo existente. No se guardará ningún pago mientras falten datos válidos.

6. Consultar pagos

Mostrar todos los pagos con identificador, nombre del cliente, fecha, importe, litros y combustible. Ordenarlos por fecha descendente y, a igualdad de fecha, por identificador descendente. Si no hay pagos, indicarlo.

7. Conservar los datos entre ejecuciones

- Utilizar obligatoriamente formato **CSV**, en uno o varios ficheros de texto UTF-8. Decidir y documentar los nombres y la ubicación de los ficheros, la distribución de los datos, el orden de las columnas, el separador, la presencia o ausencia de cabeceras, las reglas de escapado y la representación de fechas y cantidades. No se impone una estructura CSV común a todos los alumnos. El formato elegido deberá permitir recuperar sin pérdidas todos los datos admitidos por la aplicación.
- Utilizar **Files** para crear, leer y escribir los ficheros. Se permite apoyarse en otras clases de la biblioteca estándar, como **Path** y lectores o escritores obtenidos mediante **Files**.
- En el primer arranque, crear el almacenamiento necesario sin exigir la preparación manual de ficheros.
- Al volver a arrancar, recuperar los datos anteriores sin borrarlos ni duplicarlos.
- Guardar cada alta o pago antes de confirmar su éxito al usuario; no esperar a la opción de salida.
- Permitir que los textos admitidos por el programa se recuperen correctamente, incluso si contienen caracteres que tengan un significado

especial en el formato elegido.

- Ante un error de acceso a ficheros, mostrar un mensaje comprensible y no anunciar como guardada una operación fallida. Se permite finalizar de forma controlada.
- Si se detecta un fichero con registros que no se pueden interpretar, informar y finalizar de forma controlada, sin sobrescribirlo ni descartar silenciosamente sus datos.

No se exige acceso simultáneo desde varios procesos ni recuperación ante un corte de corriente durante una escritura.

Ejemplos de ejecución

Los ejemplos son independientes salvo donde se indica continuidad. Los mensajes pueden variar, pero deben transmitir la misma información. No se exige reproducir exactamente los espacios o las líneas de las tablas.

Alta y matrícula duplicada

Con el almacenamiento inicialmente vacío:

```
Opción: 1
Nombre: Ana López
Teléfono: 600123456
Matrícula: 1234abc
Cliente registrado con ID 1.

Opción: 1
Nombre: Luis Pérez
Teléfono: 600999888
Matrícula: 1234ABC
Esa matrícula ya está registrada. No se ha creado el cliente.

Opción: 2
ID      NOMBRE      TELÉFONO      MATRÍCULA
1      Ana López    600123456    1234ABC
```

Búsqueda parcial

Con Ana López y matrícula **1234ABC** registrada:

```
Opción: 3
Texto que buscar: aBc
ID      NOMBRE      TELÉFONO      MATRÍCULA
```

```
1    Ana López    600123456    1234ABC
```

Opción: 3

Texto que buscar: inexistente

No se han encontrado clientes.

Pago con corrección de datos

Con Ana López registrada como cliente 1 y sin pagos previos:

Opción: 4

```
ID    NOMBRE      TELÉFONO    MATRÍCULA
1    Ana López    600123456    1234ABC
```

ID del cliente: 1

Fecha (dd/MM/aaaa; vacío para hoy): 31/02/2026

La fecha no es válida.

Fecha (dd/MM/aaaa; vacío para hoy): 11/09/2026

Importe (€): -5

Introduce una cantidad mayor que cero y con un máximo de dos decimales.

Importe (€): 40,50

Litros: 25

Combustible: Gasolina 95

Pago 1 registrado para Ana López: 40.50 €.

Opción: 5

```
ID    CLIENTE      FECHA      IMPORTE    LITROS    COMBUSTIBLE
1    Ana López    11/09/2026    40.50 €    25.00    Gasolina 95
```

Cliente inexistente

Con Ana López como único cliente:

Opción: 4

```
ID    NOMBRE      TELÉFONO    MATRÍCULA
1    Ana López    600123456    1234ABC
```

ID del cliente: 99

No existe un cliente con ese identificador. No se ha registrado el pago.

Persistencia tras reiniciar

Después del alta y del pago de los ejemplos anteriores:

Opción: 0

Hasta pronto.

Al ejecutar de nuevo el programa, las opciones 2 y 5 deben mostrar a Ana y su pago. Al dar de alta otro cliente con matrícula diferente, recibirá el identificador 2. El siguiente pago recibirá también el identificador 2, al ser numeraciones independientes.

Calidad del código

Se valorará:

- **Modularidad:** responsabilidades y métodos con un propósito claro, evitando concentrar toda la aplicación en un único método.
- **Poco acoplamiento:** limitar las dependencias innecesarias y la propagación de cambios, especialmente cuando cambie la representación de los datos en los ficheros.
- **Ausencia de duplicación:** reutilizar la lógica común cuando tenga sentido.
- **Legibilidad:** nombres expresivos, formato consistente y uso adecuado de las herramientas del lenguaje.
- **Comentarios útiles:** explicar decisiones, restricciones o partes poco evidentes; no limitarse a repetir lo que ya dice el código.
- **Gestión de recursos y errores:** cerrar los recursos que lo requieran y responder de forma controlada a entradas incorrectas y problemas de almacenamiento.
- **Coherencia:** aplicar y explicar las decisiones de diseño acordadas en clase.

El número de clases o el uso de un patrón concreto no otorgan puntos por sí mismos. Las decisiones deben ser proporcionales al problema y facilitar la comprensión y el mantenimiento.

Entrega y trabajo en clase

Entrega

1. El código fuente Java.
2. Un README.md con la versión del JDK, instrucciones de compilación y ejecución, ubicación y formato de los ficheros y decisiones de diseño tomadas.

3. Una breve explicación de cómo has aplicado las indicaciones de diseño trabajadas en clase y qué partes tendrías que cambiar si se modificara el formato de almacenamiento.

Evaluación

Apartado	Peso
Funcionalidades y reglas de validación	20 %
Persistencia correcta y uso de <code>Files</code>	30 %
Modularidad, poco acoplamiento y legibilidad	40 %
Comentarios, documentación y pruebas	10 %